Department of Computer Science and Engineering

CSA0609 – Design and Analysis of Algorithms

Course Code & Name	CSA0609 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration / Mode	3 Days / Individual
Student Reg. No.	192511149
Student Name	Devi Shree.S

1. Part (a) – Understand the Problem

For every module the table fixes the **input size**, the **basic operation** (the operation executed most often, whose count drives the running time), **where it occurs**, and **how its count grows**. The justification paragraphs follow the table.

Module	Input size	Basic operation and where it occurs	Expected growth
M1 Duplicate detector	n = number of products	Comparison Product[i].title = Product[j].title, in the body of the inner FOR (the seller test runs only when titles match)	Quadratic: every unordered pair → $n(n-1)/2$
M2 Rate limiter	n = number of requests in the audit range	Audit(Request[j]) in the inner FOR (declared as the basic operation)	Linear: windows 1, 2, 4 … add up to about n
M3-S1 Sequential search	n = catalogue size	Comparison of the key with Product[k] inside the scan loop	Linear: up to n comparisons
M3-S2 Binary search	n = size of the sorted index	Comparison of the key with the middle entry; one per halving	Logarithmic: $\lfloor \log_2 n \rfloor + 1$
M4 Recommendation engine	n = customer–product records	Combining one partial result in the linear merge (cost f(n) = n per call); the total is dominated by the n^2 leaf subproblems created by 4 recursive calls	Current: quadratic. Redesign (2 calls): n log n
M5 Sales analytics	n = sales records	Record comparison in the sort done at every level, f(n) = n log n per call, over log n levels	$n \log^2 n$
M5 Report generator	n = records reported	The dominant term of $f(n) = 3n^2 + 10n \log_2 n + 50$, i.e. the quadratic work	Quadratic: $3n^2$

Justification, module by module

- **M1.** The loops visit each pair (i, j) with i < j exactly once and the body always evaluates the title comparison, so it is executed once per pair regardless of the data. The seller comparison is evaluated only when the titles are equal (short-circuit AND), so it is bounded by the number of title matches and cannot dominate. Titles are bounded-length strings, so one comparison is O(1).
- **M2.** Audit is the only work inside the inner loop, and nothing else in the routine costs more than O(1) per outer iteration. Its count is the sum of the window sizes 1 + 2 + 4 + … , a geometric series that is dominated by its last term, so we expect growth proportional to n, not n log n.

- **S1 / S2.** Each probe is one key comparison, and nothing else depends on n. S1 makes one comparison per examined element; S2 makes one per halving, so the counts are n and log n respectively.
- **M4.** The cost f(n) = n is the linear merge. With four sub-calls of size n/2 the recursion tree widens faster than the problem shrinks (4 calls vs a halving), so the number of leaves, $n^{(\log_2 4)} = n^2$, dominates; with two sub-calls the tree has n leaves and every level costs n, giving n log n.
- **M5.** Re-sorting at every level costs n log n per level (a sort of n records, not an O(n) merge). The per-level cost shrinks only because the sub-problem shrinks: at level i it is $n(\log n - i)$. Summed over log n levels this is of order $n \log^2 n$.
- **Report generator.** $f(n) = 3n^2 + 10n \log_2 n + 50$ is given as an operation count; the n^2 term dominates for all large n, so the basic operation is whatever statement is executed $3n^2$ times (almost certainly a nested loop over the data).

Preview of the problem areas found in this audit

• M1 ($\Theta(n^2)$), M4-current ($\Theta(n^2)$) and the report generator ($\Theta(n^2)$) will break the 10-minute window at $n = 10^6$.

• M2 is **not** a problem – the team member's analysis is wrong, the routine is linear.

• M5 ($n \log^2 n$) and S2 ($\log n$) are comfortably inside the window.

2. Part (b) – Analyse the Iterative Modules

2.1 Module M1 – Duplicate-Listing Detector

Step 1 – Explain the module

M1 is a brute-force pairwise scan. The outer loop fixes product i (0 … n−2); the inner loop compares it with every later product j (i+1 … n−1). Equal title **and** equal seller means a duplicate, and the later listing j is marked. There is no early exit and no data-dependent branch around the comparison, so the number of comparisons is identical for every input.

Step 2 – Exact number of comparisons

Basic operation: the title comparison. For a fixed i the inner loop runs $(n-1)-(i+1)+1 = n-1-i$ times:

$$C(n) = \Sigma_{i=0}^{n-2} \Sigma_{j=i+1}^{n-1} 1 = \Sigma_{i=0}^{n-2} (n-1-i)$$

Substitute $k = n-1-i$; as i runs 0 … n−2, k runs n−1 … 1:

$$C(n) = \Sigma_{k=1}^{n-1} k = (n-1)n/2 = n^2/2 - n/2$$

Check: $n = 5 \rightarrow 4+3+2+1 = 10 = 5 \cdot 4/2$ ✓. The instrumented Python routine returned exactly $n(n-1)/2$ for n = 2, 5, 10, 50, 100, 400 (Appendix A).

Step 3 – Time and space complexity

Upper bound: $n^2/2 - n/2 \le \frac{1}{2} \cdot n^2$ for $n \ge 1$. Lower bound: for $n \ge 2$, $n/2 \le n^2/4$, hence $n^2/2 - n/2 \ge n^2/2 - n^2/4 = \frac{1}{4} \cdot n^2$. Therefore $\frac{1}{4} \cdot n^2 \le C(n) \le \frac{1}{2} \cdot n^2$ for $n \ge 2$, i.e. $\mathbf{C(n) \in \Theta(n^2)}$ with $c_1 = \frac{1}{4}$, $c_2 = \frac{1}{2}$, $n_0 = 2$. Because the count does not depend on the data, best = average = worst = $\Theta(n^2)$.

Space: only the counters i and j (and the temporaries of the comparison) are used, so the auxiliary space is **Θ(1)**. MarkDuplicate flags a product in place; if a separate list of duplicates is kept it adds $O(d) \le O(n)$, which does not change the time bound.

Step 4 – Discuss the problem

n	Comparisons n(n−1)/2	Time at 10^8 ops/s	Within 10-min window?
10^5 (today)	4,999,950,000 ($\approx 5.0 \times 10^9$)	50.0 s	Yes
10^6 (next year)	499,999,500,000 ($\approx 5.0 \times 10^{11}$)	4,999.995 s = 83.3 min	No – 8.3× over the window

Ten times more products means exactly ≈ 100 times more work: the nightly job that takes 50 seconds today will take almost an hour and a half next year and will not finish inside its window. The problem is structural: the algorithm re-examines every pair although a duplicate is defined by equality of a (title, seller) key.

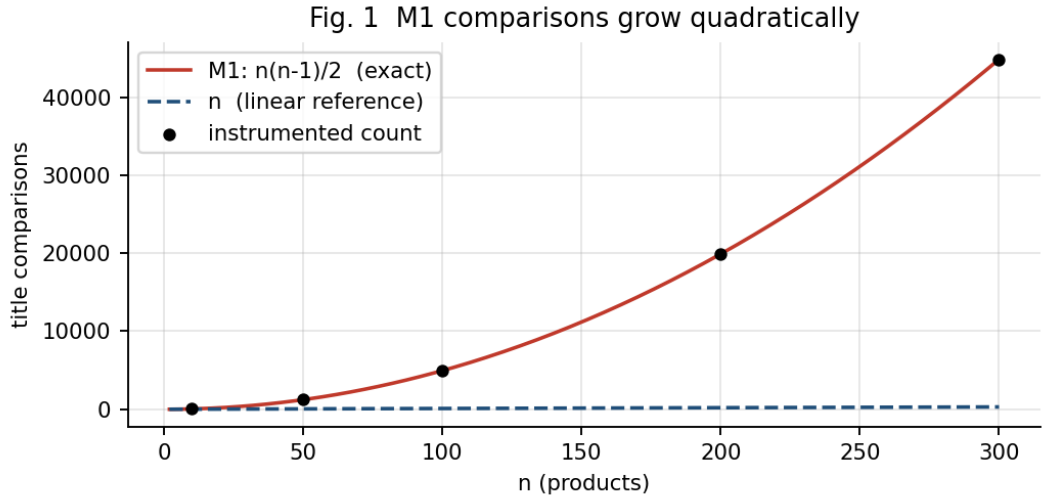


Fig. 1 – M1: exact comparisons (red) versus a linear reference; dots are instrumented counts.

Step 5 – Redesign (selected module M1′: hash-set duplicate detector)

Root cause: pairwise comparison to test equality of a key. **Correction:** remember the keys already seen in a hash set; each product then needs one expected-O(1) lookup instead of up to n comparisons.

```
seen ← empty hash set
FOR i ← 0 TO n − 1 DO
   key ← (normalise(Product[i].seller), normalise(Product[i].title))
   IF key IN seen THEN MarkDuplicate(i) ELSE INSERT(seen, key)
END FOR
```

- **Correctness:** a later listing is flagged iff an earlier listing with the same key exists – exactly the set that M1 marks (M1 marks j when some i < j matches).
- **Complexity:** n lookups/insertions, expected O(1) each → time **Θ(n)** expected, space **Θ(n)** for the set. Trade-off: memory for time. If memory is tight, sort by (seller, title) and compare neighbours: Θ(n log n) time, O(1) extra space with an in-place sort. In production a database UNIQUE index on (seller, title) detects the duplicate at upload time in O(log n).
- **Hash-collision caution:** use a randomised/universal hash so an adversarial seller cannot force O(n) buckets.

Fig. 8 Empirical proof that the M1 redesign is linear

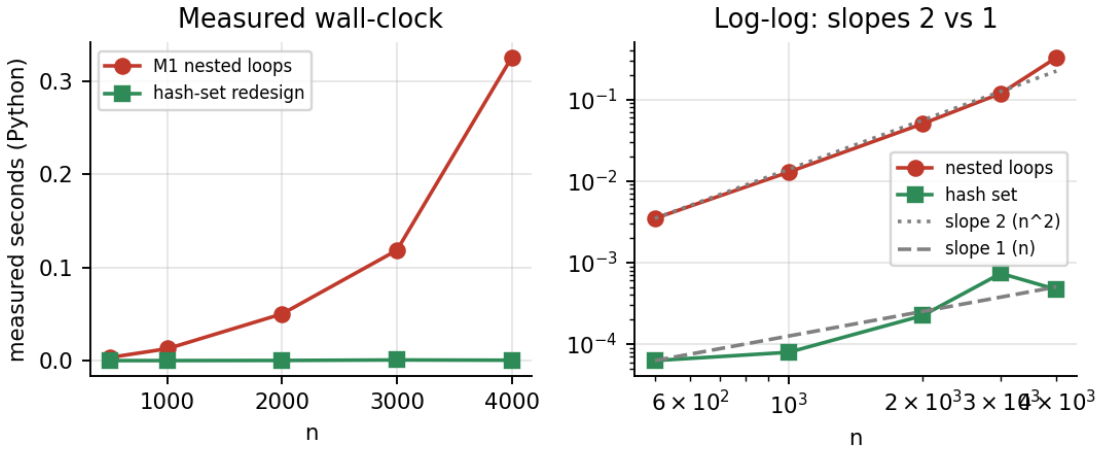


Fig. 2 – Measured Python wall-clock of M1 and the hash redesign (same data); fitted log-log slopes ≈ 2.1 and ≈ 1.2.

n	M1 nested loops (s)	Hash redesign (s)	Speed-up
500	0.0035	0.00006	55×
1,000	0.0129	0.00008	162×
2,000	0.0503	0.00022	225×
3,000	0.1186	0.00074	161×
4,000	0.3254	0.00047	695×

Measured evidence: the nested loops follow a slope of 2.11 on log-log axes (quadratic), the hash version 1.19 (linear, with timing noise at tiny n). The redesign handled $n = 10^6$ products in about 0.36 s of Python time, whereas the quadratic version would need $\approx 5 \times 10^{11}$ comparisons.

2.2 Module M2 – API Rate Limiter

Step 1 – Explain the module

The outer loop keeps a window size i that doubles each time (1, 2, 4, 8, …) while i < n. For each window size the inner loop audits requests 0 … i−1, i.e. exactly **i** requests, not n.

Step 2 – Exact number of Audit operations

The while-loop body executes for $i = 2^0, 2^1, \ldots, 2^{(K-1)}$, where K is the smallest integer with $2^K \geq n$, so $\mathbf{K = \lceil \log_2 n \rceil}$ iterations. In the t-th iteration (t = 0 … K−1) the inner loop performs 2^t Audit operations:

$$A(n) = \Sigma\ (t = 0 \text{ to } K-1)\ 2^t = 2^K - 1 = 2^{\lceil \log_2 n \rceil} - 1$$

(geometric series: $\Sigma\ 2^t = (2^K - 1)/(2 - 1)$). When n is a power of two, $K = \log_2 n$ and $\mathbf{A(n) = n - 1}$. For any $n \geq 2$, $n \leq 2^K < 2n$ and 2^K is even, so $2^K \leq 2n - 2$:

$$n - 1 \leq A(n) \leq 2n - 3$$

Step 3 – Is the Θ(n log n) claim correct? No.

The team member multiplied "log n outer iterations" by "up to n inner iterations". That is a valid **upper bound** (A(n) = O(n log n)), but it is far from tight because the inner loop does **not**

run n times in every outer iteration – it runs i times, and i doubles. The sum is a geometric series dominated by its last term, about n/2 … n.

Tight bound – formal proof. For $n \geq 2$: $A(n) \geq n - 1 \geq n/2$ and $A(n) \leq 2n - 3 \leq 2n$. So $\frac{1}{2} \cdot n \leq A(n) \leq 2 \cdot n$, i.e. **$A(n) \in \Theta(n)$** with $c_1 = \frac{1}{2}$, $c_2 = 2$, $n_0 = 2$.

Why $\Theta(n \log n)$ fails: $A(n)/(n \log_2 n) \leq 2n/(n \log_2 n) = 2/\log_2 n \to 0$ as $n \to \infty$. A function whose ratio to n log n tends to 0 is $o(n \log n)$, hence not $\Omega(n \log n)$ and therefore not $\Theta(n \log n)$.

Step 4 – Verification for n = 1,024

$n = 1{,}024 = 2^{10}$, so K = 10 iterations (i = 1 … 512; i = 1,024 fails $i < n$).

Outer iteration t	i = 2^t	Audit calls this iteration	Running total
0	1	1	1
1	2	2	3
2	4	4	7
3	8	8	15
4	16	16	31
5	32	32	63
6	64	64	127
7	128	128	255
8	256	256	511
9	512	512	1,023

Total = 1 + 2 + 4 + … + 512 = **1,023** = $2^{10} - 1 = n - 1$ ✓ (also confirmed by the instrumented code). The claimed $n \log_2 n = 1{,}024 \times 10 = 10{,}240$ is ten times larger than the true count, and the gap grows with n: for $n = 10^6$ the true count is 2^20 − 1 = **1,048,575** against a claimed $\approx 1.99 \times 10^7$.

Fig. 2 M2 is Theta(n), not Theta(n log n)

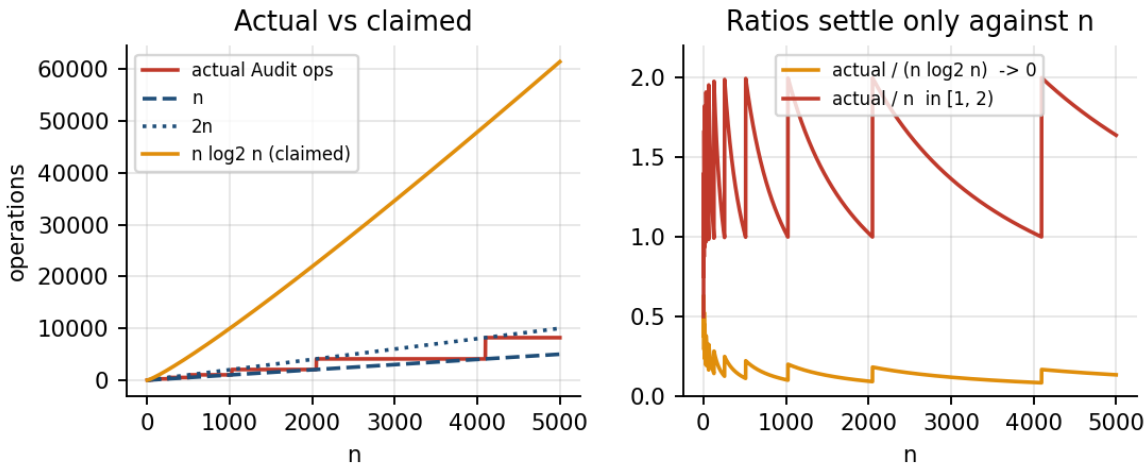


Fig. 3 – M2: actual Audit operations stay between n and 2n; the ratio to $n \log_2 n$ tends to 0.

Step 5 – Space, problem, redesign

- **Space:** $\Theta(1)$ – only i and j. **Time:** $\Theta(n)$.
- **Problem?** None in performance: 1.05×10^6 operations at $n = 10^6$ is 0.0105 s. The only "issue" was the wrong analysis, which could have caused an unnecessary redesign. **Decision: keep M2 unchanged** – a routine that must touch about n requests cannot be asymptotically faster than $\Theta(n)$.

3. Part (c) – Solve a Recurrence by Substitution (Search S2)

Step 1 – Explain the module

S2 is binary search on a **sorted** product index. Each probe compares the key with the middle entry (one comparison) and discards half, leaving a problem of size n/2: **T(n) = T(n/2) + 1, T(1) = 1.** Let n = 2^k (the general case follows below).

Step 2 – Backward substitution, every step

$$
\begin{aligned}
T(n) &= T(n/2) + 1 && \text{(original)} \\
&= [\, T(n/4) + 1 \,] + 1 = T(n/4) + 2 && \text{(replace } T(n/2) = T(n/4) + 1) \\
&= [\, T(n/8) + 1 \,] + 2 = T(n/8) + 3 && \text{(replace } T(n/4) = T(n/8) + 1) \\
&= [\, T(n/16) + 1 \,] + 3 = T(n/16) + 4 \\
&\;\vdots \\
&= T(n/2^i) + i && \text{(pattern after } i \text{ substitutions)}
\end{aligned}
$$

The recursion stops when n/2^i = 1, i.e. 2^i = n, i = $\log_2 n$. Substituting and using T(1) = 1:

$$T(n) = T(1) + \log_2 n = 1 + \log_2 n$$

Proof of the pattern by induction on i: i = 1: T(n) = T(n/2) + 1 ✓. Assume T(n) = T(n/2^i) + i. Apply the recurrence to T(n/2^i) = T(n/2^(i+1)) + 1 to get T(n) = T(n/2^(i+1)) + (i + 1) ✓.

General n (not a power of two): the discarded half has ⌊n/2⌋ entries, so T(n) = T(⌊n/2⌋) + 1 and the exact closed form is **T(n) = ⌊$\log_2 n$⌋ + 1** (the number of bits of n).

n	1	2	8	1,023	1,024	10^5	10^6
T(n) = ⌊$\log_2 n$⌋ + 1	1	2	4	10	11	17	20

For n = 1,023 the instrumented binary search needed at most 10 comparisons and 9.0098 on average (Appendix A).

Step 3 – Time and space complexity of both implementations

Implementation	Time	Auxiliary space	Why
Iterative (loop with lo, hi, mid)	Θ(log n)	Θ(1)	Three index variables; no stack growth
Recursive (search(arr, lo, hi, key))	Θ(log n)	Θ(log n)	Recursion depth ⌊$\log_2 n$⌋ + 1 frames of O(1) each (≈ 20 frames for n = 10^6); the stack is not freed until the base case unless the compiler does tail-call optimisation

Pass indices (lo, hi), never array slices: copying the sub-array would cost O(n) per call and destroy the logarithmic bound. Recommendation for production: the **iterative** version – same speed, constant memory, no stack-overflow risk.

Fig. 3 Search versions S1 and S2

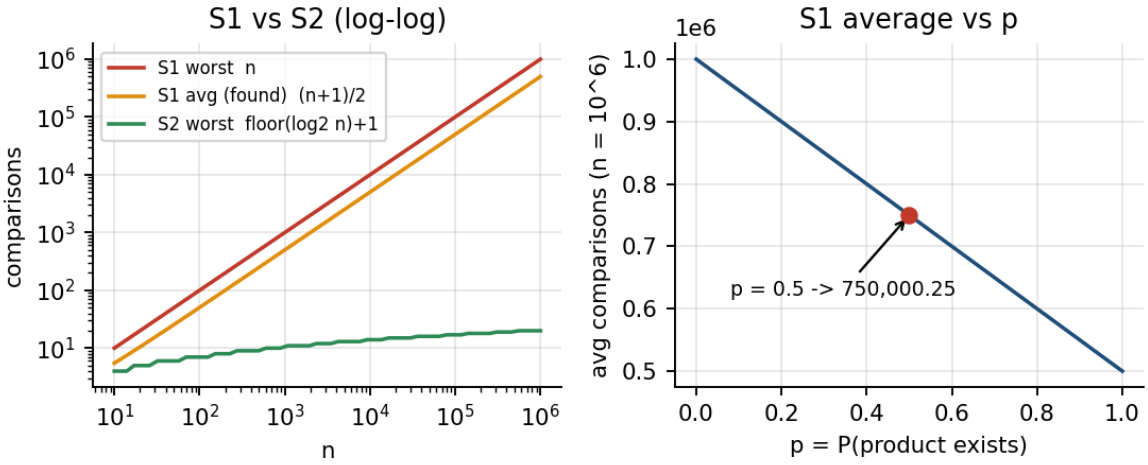


Fig. 4 – Left: S1 vs S2 comparison counts (log-log). Right: S1 average cost versus p ($n = 10^6$). Used in Parts (c) and (e).

4. Part (d) – Apply the Master Theorem

Master Theorem for T(n) = a·T(n/b) + f(n), a ≥ 1, b > 1: compare f(n) with n^(log_b a). **Case 1:** f(n) = O(n^(log_b a − ε)) for some ε > 0 ⇒ Θ(n^(log_b a)). **Case 2:** f(n) = Θ(n^(log_b a)) ⇒ Θ(n^(log_b a) · log n). **Case 3:** f(n) = Ω(n^(log_b a + ε)) and a·f(n/b) ≤ c·f(n) for some c < 1 ⇒ Θ(f(n)).

4.1 Module M4 – Recommendation Engine

Step 1 – Explain the module

The engine splits the customer–product data and recurses on the parts, merging in linear time. Replicated partitions force **four** sub-problems of size n/2 instead of two.

Step 2 – Current design: T(n) = 4T(n/2) + n

Quantity	Value	Quantity	Value
a	4	b	2
f(n)	n	n^(log_b a) = n^($\log_2$ 4)	n^2
Comparison	f(n) = n = O(n^(2 − ε)) with ε = 1	Case used	Case 1 (leaves dominate)

T(n) = Θ(n^($\log_2$ 4)) = Θ(n^2)

Cross-check by recursion tree. Level i has 4^i nodes of size n/2^i, costing 4^i · n/2^i = n·2^i; the cost per level doubles. Over levels 0 … m−1 (m = $\log_2$ n): Σ n·2^i = n(2^m − 1) = n^2 − n; the 4^m = n^2 leaves cost T(1) = 1 each: **T(n) = (n^2 − n) + n^2 = 2n^2 − n** ✓ (identical to the recurrence evaluated numerically for n = 1 … 2^{15}).

Step 3 – Proposed redesign: T(n) = 2T(n/2) + n

Quantity	Value	Quantity	Value
a	2	b	2
f(n)	n	n^(log_b a) = n^($\log_2$ 2)	n
Comparison	f(n) = Θ(n) = Θ(n^($\log_2$2) · $\log^0$ n)	Case used	Case 2 (k = 0, all levels equal)

$$T(n) = \Theta(n^{(\log_2 2)} \cdot \log n) = \Theta(n \log n)$$

Recursion tree: every level costs 2^i · n/2^i = n and there are $\log_2$ n levels plus n leaves: **T(n) = n $\log_2$ n + n**.

Step 4 – What happens when the data doubles

$$\text{Current:}\quad T(2n)/T(n) = (8n^2 - 2n)/(2n^2 - n) = 4 + 2/(2n - 1) \;\to 4$$

$$\text{Redesign:}\quad T(2n)/T(n) = (2n \log_2 n + 4n)/(n \log_2 n + n) = 2 + 2/(\log_2 n + 1) \;\to 2$$

Data change	Current 4T(n/2) + n	Redesign 2T(n/2) + n
Double (n → 2n)	× 4.00 (e.g. 4.0000019 at n = 2^{19})	× 2.10 at n = 2^{19} (2.33 at n = 32)

Data change	Current 4T(n/2) + n	Redesign 2T(n/2) + n
10 × more data ($10^5 \rightarrow 10^6$)	× ≈ 100	× ≈ 11.9
n = 10^6 operations	2 × 10^{12} (5.56 h)	2.09 × 10^7 (0.21 s)

Interpretation: with replication, doubling the catalogue quadruples the work; the redesign roughly doubles it. At n = 10^6 the redesign is ≈ 95,500 times faster. **Decision: adopt the proposed redesign (remove replicated partitions).**

Fig. 4 M4: replication makes the engine quadratic

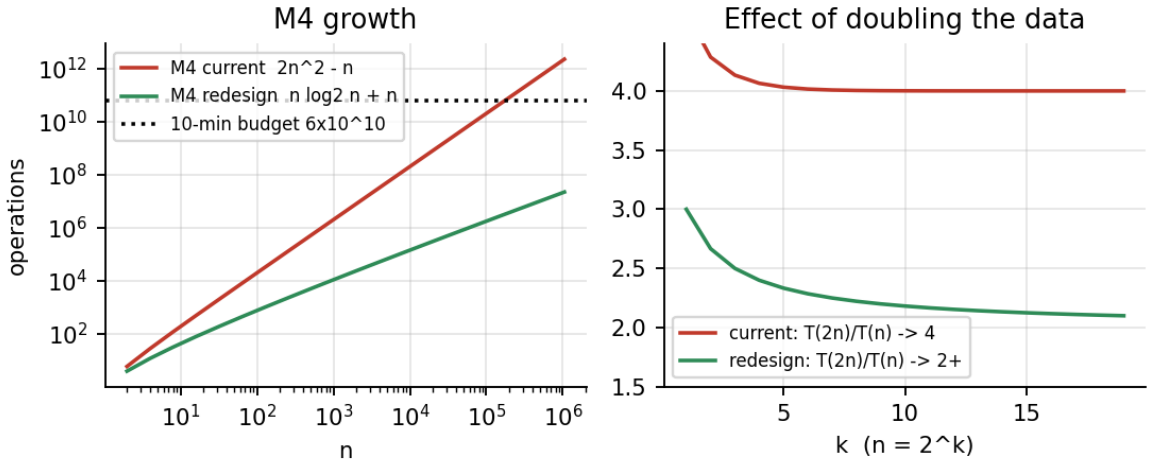


Fig. 5 – M4: operations versus n (left) and cost multiplier when the data doubles (right).

4.2 Module M5 – Sales Analytics Engine: T(n) = 2T(n/2) + n log n

Step 1 – Explain the module

The data is halved recursively, but instead of an O(n) merge the engine **fully sorts** its n records by revenue at every level, which costs f(n) = n log n per call.

Step 2 – Does a standard case apply? No

Here a = 2, b = 2, so n^(log_b a) = n^($\log_2$ 2) = n and f(n) = n log n.

Case	Requirement	Test on f(n) = n log n	Applies?
1	f = O(n^(1 − ε))	n log n is larger than n, so not O(n^(1−ε))	No
2 (basic)	f = Θ(n)	n log n / n = log n → ∞, so f ≠ O(n)	No
3	f = Ω(n^(1+ε)), ε > 0	n log n / n^(1+ε) = log n / n^ε → 0 for every ε > 0	No

f(n) is larger than n^(log_b a) but only by a logarithmic factor, not by a polynomial factor, so it falls in the "gap" between Cases 2 and 3.

Step 3 – Extended Master Theorem

Extended Case 2: if f(n) = Θ(n^(log_b a) · log^k n) with k ≥ 0, then T(n) = Θ(n^(log_b a) · log^(k+1) n). Here f(n) = n log n = n · $\log^1$ n, so k = 1:

$T(n) = \Theta(n \cdot \log^2 n)$

Step 4 – Derivation by recursion tree (exact)

Level i (i = 0 … m−1, m = $\log_2 n$) has 2^i nodes of size n/2^i. Each costs (n/2^i)·$\log_2$(n/2^i) = (n/2^i)(m − i), so the level costs 2^i · (n/2^i)(m − i) = **n(m − i)**:

$$T(n) = \Sigma\ (i = 0 \text{ to } m-1)\ n(m - i) + n \cdot T(1) = n \cdot m(m + 1)/2 + n$$

Because m(m+1)/2 = Θ(m²) = Θ($\log^2$ n), T(n) = Θ(n $\log^2$ n) ✓. Numerical check: the exact value matches the recurrence for n = 1 … 2^{15}, and T(n)/(n $\log^2$ n) → ½ (Fig. 6). Unlike Case 2, the levels do not cost the same: the top level costs n·m and the bottom non-trivial level only n, an arithmetic progression – which is why one extra log factor appears.

Fig. 5 M5: T(n) = 2T(n/2) + n log n is Theta(n log^2 n)

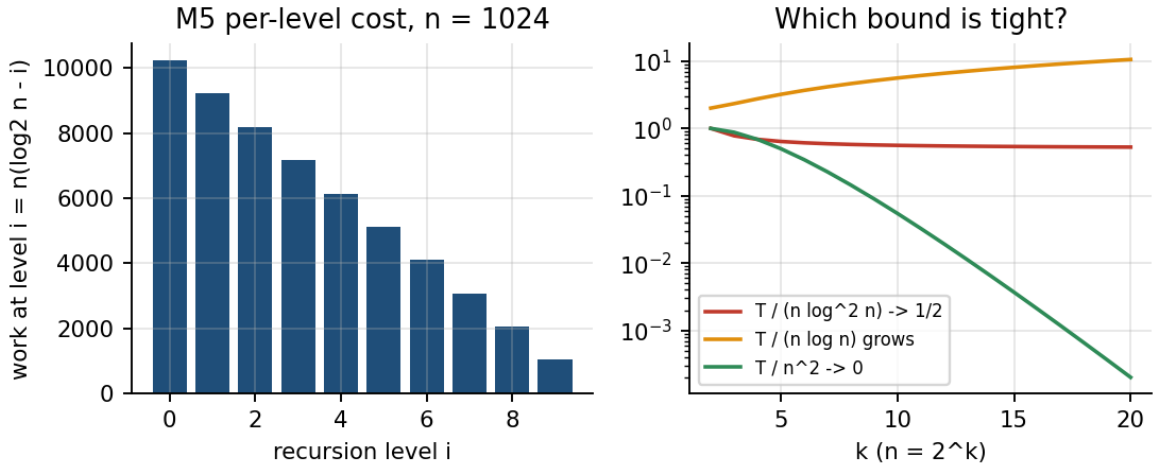


Fig. 6 – M5: cost per recursion level (left); T/(n $\log^2$n) converges to ½ while the other normalisations drift (right).

Step 5 – Problem and redesign

At n = 10^6: m = 19.93, T ≈ 10^6 · (19.93 · 20.93 / 2) + 10^6 ≈ **2.10 × 10^8 operations ≈ 2.1 s** – inside the window. Still, re-sorting at every level is wasteful: the halves are already sorted after the recursive calls, so an **O(n) merge** (the merge-sort idea) gives T(n) = 2T(n/2) + n = Θ(n log n) (Case 2) ≈ 2.09 × 10^7 operations, i.e. about ten times faster (the saved factor is ($\log_2$ n + 1)/2 ≈ 10.5). Priority: low (optional tune-up).

5. Part (e) – Best, Average and Worst Cases

5.1 S1 – Sequential search of an unsorted catalogue

Case	Input condition	Comparisons	$n = 10^6$
Best	Key is the first element	1	1
Worst	Key is the last element, or is absent	n	1,000,000
Average (found, uniform position)	Each of n positions equally likely	(n + 1)/2	500,000.5
Average (found with probability p)	See derivation below	p(n + 1)/2 + n(1 − p)	750,000.25 at p = 0.5

Derivation of the average with probability p

The product exists with probability p, and given that, it is equally likely to sit at any of the n positions (probability p/n each, costing i comparisons for position i). With probability (1 − p) it is absent and the scan makes all n comparisons:

$$A(n, p) = \Sigma\ (i = 1 \text{ to } n)\ i \cdot (p/n) + n(1 - p) = (p/n) \cdot n(n + 1)/2 + n(1 - p)$$

$$A(n, p) = p(n + 1)/2 + n(1 - p) = n(1 - p/2) + p/2$$

Check: p = 1 gives (n + 1)/2 (successful search); p = 0 gives n (unsuccessful). **Evaluation:** n = 1,000,000, p = 0.5:

$$A = 0.5 \times (1{,}000{,}001)/2 + 1{,}000{,}000 \times 0.5 = 250{,}000.25 + 500{,}000 = 750{,}000.25$$

p	0	0.25	0.5	0.75	1
$A(10^6, p)$	1,000,000	875,000.125	750,000.25	625,000.375	500,000.5

A Monte-Carlo check with n = 1,000, p = 0.5 over 40,000 random searches gave 748.9 comparisons against the formula's 750.25 ✓. All three S1 cases are Θ(n) except the best case Θ(1); the **average is Θ(n)** for every constant p, so S1 does not scale.

5.2 S2 – Binary search of the sorted index

Case	Input condition	Comparisons	$n = 10^6$
Best	Key equals the middle entry of the whole index	1	1
Worst	Key found only at the deepest level, or absent	$\lfloor \log_2 n \rfloor + 1$	20
Average (found)	All n keys equally likely	$\approx \log_2 n - 1$ (exactly ((k−1)2^k + 1)/(2^k − 1) for n = 2^k − 1)	18.95
Average (absent)	All n + 1 gaps equally likely	$\approx \log_2 n$ ($\approx \lfloor \log_2 n \rfloor + 1$ for almost every gap)	19.95

Derivation of the average (found), n = 2^k − 1: the probe tree is a complete binary tree. Level l (l = 1 … k) holds 2^(l−1) keys, each found after l comparisons:

$$\text{Total} = \Sigma\ (l = 1 \text{ to } k)\ l \cdot 2^{(l-1)} = (k - 1) \cdot 2^k + 1, \quad \text{Average} = ((k - 1)2^k + 1)/(2^k - 1) \approx k - 1$$

Example n = 1,023 (k = 10): (9 × 1,024 + 1)/1,023 = 9.0098, equal to the value measured by the code. For n = 10^6 an exact count over all keys gives 18.95. So the average case is only about one comparison better than the worst: **best Θ(1), average Θ(log n), worst Θ(log n).**

Comparison for n = 10^6 (one query)

S1 average (p = 0.5): 750,000 comparisons = 7.5 ms. S2 average: ≈ 19 comparisons = 0.19 µs. Ratio ≈ 39,000 : 1.

Precondition of S2: the index must stay sorted (use a B-tree / balanced index so insertions cost O(log n), not O(n) array shifting).

6. Part (f) – Prove an Asymptotic Bound

Definition. $f(n) \in \Theta(g(n))$ iff there exist constants $c_1, c_2 > 0$ and $n_0 \geq 1$ such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$. Here $f(n) = 3n^2 + 10n \log_2 n + 50$ and $g(n) = n^2$.

Step 1 – Lower bound ($f \in \Omega(n^2)$)

For $n \geq 1$ both $10n \log_2 n \geq 0$ and $50 > 0$, so $f(n) \geq 3n^2$. Hence $\mathbf{c_1 = 3}$ works for $\mathbf{n_0 = 1}$.

Step 2 – Upper bound ($f \in O(n^2)$)

For $n \geq 1$: $\log_2 n \leq n$, so $10n \log_2 n \leq 10n^2$; and $50 \leq 50n^2$. Then:

$$f(n) = 3n^2 + 10n \log_2 n + 50 \leq 3n^2 + 10n^2 + 50n^2 = 63n^2 \quad (n \geq 1)$$

So $\mathbf{c_2 = 63}$ works for $n_0 = 1$.

> **Conclusion**
> $3n^2 \leq f(n) \leq 63n^2$ for all $n \geq 1$, so $f(n) \in \Theta(n^2)$ with $\mathbf{c_1 = 3, c_2 = 63, n_0 = 1}$.

Step 3 – A tighter, equally valid choice

Choose $c_2 = 4$. We need $10n \log_2 n + 50 \leq n^2$. Let $g(n) = n^2 - 10n \log_2 n - 50$: $g(59) = -39.7 < 0$ but $g(60) = 3600 - 3544.1 - 50 = 5.9 > 0$. For $n \geq 60$, $g'(n) = 2n - 10 \log_2 n - 14.43 \geq 120 - 59.1 - 14.43 > 0$ and stays positive ($2n$ grows faster than $10 \log_2 n$), so g is increasing and positive. Hence $\mathbf{3n^2 \leq f(n) \leq 4n^2}$ **for** $\mathbf{n \geq n_0 = 60}$ (checked by computer for $60 \leq n \leq 200{,}000$). Limit test: $f(n)/n^2 = 3 + 10 \log_2 n/n + 50/n^2 \to 3$, a finite positive constant, confirming $\Theta(n^2)$.

Step 4 – f(n) is not O(n)

Proof by contradiction. Suppose $f(n) \in O(n)$: there are $c > 0$ and n_0 with $f(n) \leq c \cdot n$ for all $n \geq n_0$. Since $f(n) \geq 3n^2$, we would get $3n^2 \leq c \cdot n$, i.e. $n \leq c/3$, for every $n \geq n_0$. Take $n = \max(n_0, \lceil c/3 \rceil + 1)$: then $n > c/3$, a contradiction. Therefore $\mathbf{f(n) \notin O(n)}$. Equivalently $f(n)/n \geq 3n \to \infty$, so f grows strictly faster than any linear function.

Fig. 6 Report generator f(n) = 3n^2 + 10 n log2 n + 50

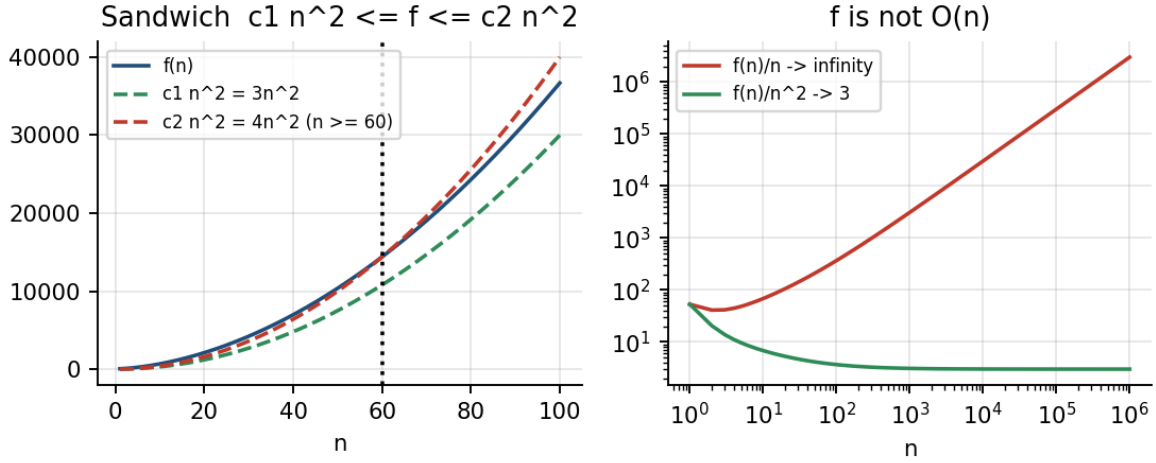


Fig. 7 – Left: $f(n)$ is sandwiched between $3n^2$ and $4n^2$ beyond $n_0 = 60$. Right: $f(n)/n^2 \to 3$ and $f(n)/n \to \infty$.

Meaning for the platform: the nightly report costs about $3n^2$ operations; the lower-order term $10n \log_2 n$ is irrelevant at scale (at $n = 10^6$ it is 2×10^8 against 3×10^{12}). Doubling the catalogue multiplies the report time by ≈ 4.

7. Part (g) – Scalability Decision

Operations are exact closed forms at $n = 10^6$ (real-valued $\log_2 10^6 = 19.9316$ for the recurrence-based modules, unit cost per f-unit and $T(1) = 1$). Time = operations ÷ 10^8. Window: 10 min = 600 s = 6×10^{10} operations.

Module	Complexity	Basic operations	Running time	Window 10 min
M1 duplicate detector	Theta(n^2)	5.000 × 10^11	1.39 h	FAIL (8.3× window)
M2 rate limiter	Theta(n)	1.049 × 10^6	0.0105 s	PASS
S2 binary search (one query)	Theta(log n)	2.000 × 10^1	0.2 µs	PASS
M4 current 4T(n/2)+n	Theta(n^2)	2.000 × 10^12	5.56 h	FAIL (33.3× window)
M4 redesign 2T(n/2)+n	Theta(n log n)	2.093 × 10^7	0.2093 s	PASS
M5 sales analytics 2T(n/2)+n log n	Theta(n log^2 n)	2.096 × 10^8	2.1 s	PASS
M5 report generator f(n)	Theta(n^2)	3.000 × 10^12	8.33 h	FAIL (50.0× window)

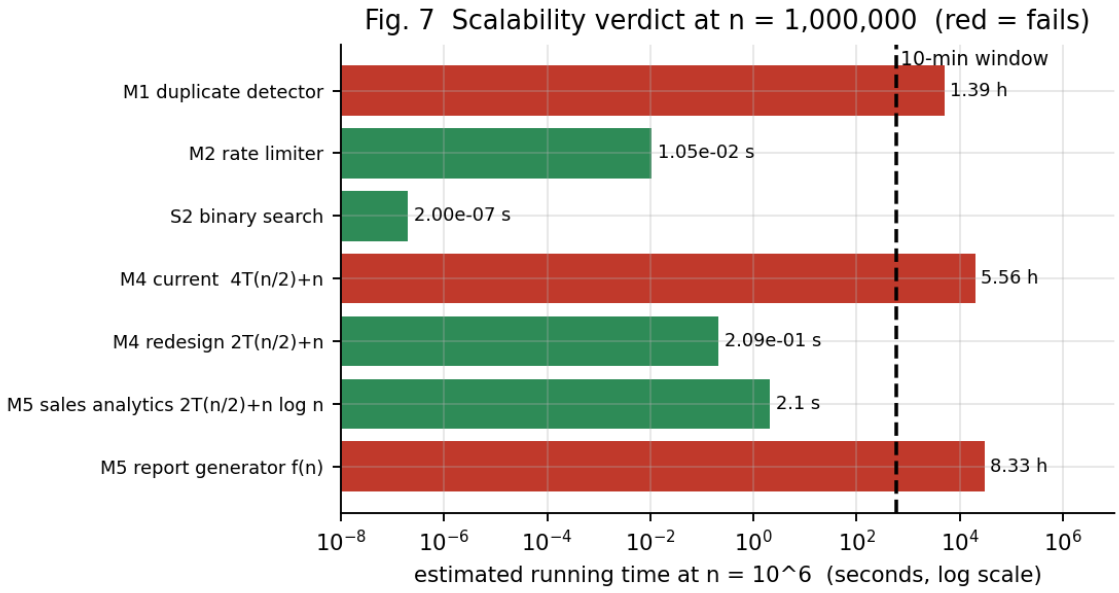


Fig. 8 – Estimated running times at $n = 10^6$; red bars exceed the 10-minute window.

Which modules fail?

- **M1 – 83.3 minutes (8.3× over).** $\Theta(n^2)$, 5×10^{11} comparisons.
- **M4 current design – 5.56 hours (33× over).** $\Theta(n^2)$ because of replication.
- **Report generator (part of M5) – 8.33 hours (50× over).** $3n^2$ dominates. This is easy to miss because it is described separately from the recursion; it is the worst offender.

- Pass: M2 (10.5 ms), S2 (0.2 µs per query), M4 redesign (0.21 s), M5 sort recursion (2.1 s).

Early-warning observation: at today's $n = 10^5$, M1 takes 50 s, M4-current 200 s and the report 300 s – each inside the window, which is why nobody noticed. A 10× growth turns all three into ×100 failures. Run back to back, the three would already consume ≈ 550 of the 600 seconds today.

Redesign recommendations with justification

Module	Defect (root cause)	Selected redesign	New cost at $n = 10^6$	Improvement
M1 – must	Pairwise scan to test key equality: $\Theta(n^2)$	Hash set on (seller, title); or sort + neighbour scan	$\Theta(n)$: 10^6 ops = 0.01 s (sort option: 2×10^7 = 0.2 s)	≈ 500,000×
M4 – must	Replication → 4 sub-problems, leaves dominate (Case 1)	Proposed design without replication, $2T(n/2) + n$ (Case 2)	$\Theta(n \log n)$: 2.09×10^7 ops = 0.21 s	≈ 95,500×
Report generator – must	Quadratic term $3n^2$ (nested pass over the records)	Replace the nested loop by a sort/hash aggregation (each group computed in one pass); target $\Theta(n \log n)$	$\approx 10n \log n = 2 \times 10^8$ ops ≈ 2 s	≈ 15,000×
M5 sort – optional	Full re-sort at every level ($\Theta(n \log^2 n)$)	Sort once / merge sorted halves in $O(n)$: $2T(n/2) + n$	$\Theta(n \log n)$: 2.09×10^7 ops = 0.21 s	≈ 10×
S1 – replace	$\Theta(n)$ per query: a batch of 10^6 queries = 10^{12} ops	Use S2 on a sorted/B-tree index (or a hash index for exact match, $O(1)$)	$\Theta(\log n)$ per query: 20 ops	≈ 50,000×
M2, S2 – keep	None (M2 linear, S2 logarithmic)	No change; correct the wrong $\Theta(n \log n)$ note in the design docs	0.0105 s / 0.2 µs	–

Fig. 9 Before vs after redesign (n = 1,000,000)

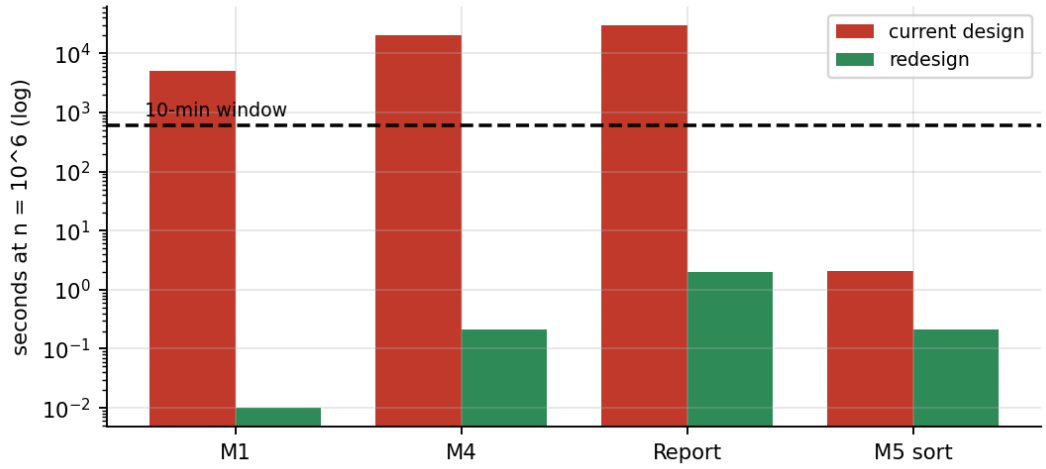


Fig. 9 – Running time at $n = 10^6$ before and after the recommended redesigns (log scale).

Justification of the decisions. (1) The three failing modules exceed the window by factors of 8–50, so micro-optimisation or faster hardware cannot save them: a 10× faster machine still leaves them 1–5× over, and the next doubling of the catalogue would quadruple the cost again. (2) Each redesign changes the **growth order** ($n^2 \rightarrow n \log n$ or n), which is the only fix that survives further growth – at $n = 10^7$ the redesigned M4 needs ≈ 2.5 s while the old one would need ≈ 23 days. (3) M2, S2 and M5 are already inside the window and are left alone (M5 only optionally tuned), so engineering effort goes where the proof shows the risk.

Final recommendation to the CTO

1. Redesign **M1**, **M4** and the **report generator** before the expansion (hash-based detection; remove replication; remove the quadratic pass).
2. Standardise on **S2** (sorted/B-tree index) for product search; retire S1 for large catalogues.
3. Keep **M2** unchanged; schedule the **M5** merge-based tune-up as low-priority work.

8. Module-wise Efficiency Scorecard (time and space)

Module	Best	Average	Worst	Space	Efficiency verdict
M1 Duplicate detector	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$ aux.	Poor – redesign
M1′ Hash set	$\Theta(n)$	$\Theta(n)$ exp.	$\Theta(n^2)$ adversarial hashing (avoidable)	$\Theta(n)$	Very good
M2 Rate limiter	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Efficient – keep
S1 Sequential search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	Weak at scale
S2 Binary search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(1)$ iter. / $\Theta(\log n)$ rec.	Excellent
M4 current	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$ buffers + $\Theta(\log n)$ stack if depth-first; up to $\Theta(n^2)$ if all replicas are held	Poor – redesign
M4 redesign	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$ + $\Theta(\log n)$ stack	Good (optimal for merge-based)
M5 sales analytics	$\Theta(n \log^2 n)$	$\Theta(n \log^2 n)$	$\Theta(n \log^2 n)$	$\Theta(n)$ + $\Theta(\log n)$ stack	Acceptable, tunable
Report generator	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	not stated	Poor – redesign

Space entries for the recursive modules are the standard textbook assumption (O(n) merge buffer, O(log n) recursion depth); the problem statement itself only fixes the time recurrences. M1′ buys its speed with Θ(n) memory; the sort-based alternative keeps O(1) extra space at Θ(n log n) time.

Appendix A – Verification Code (Python) and Outputs

The complete script (analysis.py, supplied with this document) generates every figure and number. Key excerpts:

```
import math
# ---- M1: instrumented count vs n(n-1)/2
def m1_count(n):
    cnt = 0
    for i in range(0, n-1):
        for j in range(i+1, n):
            cnt += 1                    # one title comparison
    return cnt
assert all(m1_count(n) == n*(n-1)//2 for n in (2, 5, 10, 50, 100, 400))

# ---- M2: instrumented Audit count vs 2^ceil(log2 n) - 1
def m2_count(n):
    i, tot = 1, 0
    while i < n:
        for j in range(i): tot += 1      # Audit(Request[j])
        i *= 2
    return tot
assert all(m2_count(n) == 2**math.ceil(math.log2(n)) - 1 for n in range(2, 3000))
print(m2_count(1024))                    # -> 1023

# ---- S2: binary search, one comparison per probe
def bsearch(a, key):
    lo, hi, c = 0, len(a)-1, 0
    while lo <= hi:
        mid = (lo+hi)//2; c += 1
        if a[mid] == key: return c
        lo, hi = (mid+1, hi) if a[mid] < key else (lo, mid-1)
    return c
a = list(range(1023)); print(max(bsearch(a, x) for x in a))      # -> 10

# ---- S1 average: p(n+1)/2 + n(1-p)
S1avg = lambda n, p: p*(n+1)/2 + (1-p)*n
print(S1avg(10**6, .5))                  # -> 750000.25

# ---- M4 / M5 recurrences vs closed forms (n = 2^k)
T4  = lambda n: 1 if n == 1 else 4*T4(n//2) + n
T4r = lambda n: 1 if n == 1 else 2*T4r(n//2) + n
T5  = lambda n: 1 if n == 1 else 2*T5(n//2) + n*math.log2(n)
for k in range(16):
    n = 2**k
    assert T4(n) == 2*n*n - n and T4r(n) == n*k + n
    assert abs(T5(n) - (n*k*(k+1)/2 + n)) < 1e-6*max(1, T5(n))

# ---- (f): constants for f(n) = 3n^2 + 10 n log2 n + 50
f = lambda n: 3*n*n + 10*n*math.log2(n) + 50
assert all(3*n*n <= f(n) <= 63*n*n for n in range(1, 5000))
assert all(f(n) <= 4*n*n for n in range(60, 200000)) and f(59) > 4*59*59

# ---- (g): runtime = operations / 1e8
m = math.log2(10**6); n = 10**6
print(n*(n-1)/2/1e8, (2*n*n-n)/1e8, (n*m*(m+1)/2+n)/1e8, f(n)/1e8)

# ---- M1 redesign: hash set on (seller, title)
def m1_hash(P):
    seen, d = set(), 0
```

```
  for p in P:
    if p in seen: d += 1
    else: seen.add(p)
  return d
```

Output of the verification run

Check	Result
M1 count = n(n−1)/2 for n = 2, 5, 10, 50, 100, 400	All equal (1; 10; 45; 1,225; 4,950; 79,800)
M2 count = $2^{\lceil \log_2 n \rceil} - 1$ for n = 2 … 2,999; n = 1,024	All equal; 1,023
Binary search worst comparisons for n = 2^k, k = 1 … 14	= k + 1 = $\lfloor \log_2 n \rfloor + 1$ in every case
n = 1,023: worst / average comparisons	10 / 9.0098
n = 10^6: S2 worst / average found / average absent	20 / 18.95 / 19.95
S1 formula, n = 10^6, p = 0.5; Monte-Carlo n = 1,000	750,000.25; 748.9 measured vs 750.25 predicted
M4 recurrences vs $2n^2-n$ and $n \log_2 n + n$ (n = 1 … 2^{15})	Exact match
M5 recurrence vs $n \cdot m(m+1)/2 + n$ (n = 1 … 2^{15})	Match (floating-point tolerance 10^{-6})
f(n): $3n^2 \le f \le 63n^2$ (n < 5,000); $f \le 4n^2$ for n ≥ 60 (to 2×10^5); $f(59) > 4 \cdot 59^2$	All true
M1 hash redesign, n = 10^5 and 10^6 products (Python wall-clock)	0.020 s and 0.357 s